1. **Amazon SNS — Console Dashboard Wise Important Interview Questions & Answers**

```
SNS
│
├── Dashboard
│
├── Topics
│   └── Individual Topic
│       ├── Subscriptions
│       ├── Access Policy
│       ├── Encryption
│       ├── Delivery Status
│       └── Message Publishing
│
├── Subscriptions
├── Phone Numbers
├── Platform Applications
├── Mobile Push Notifications
└── Events / Monitoring
```

Dashboard

1. What is Amazon SNS?

Answer:

Amazon SNS (Simple Notification Service) is a managed messaging service used to send notifications and messages to multiple subscribers.

It follows a publish-subscribe model where a publisher sends a message to an SNS topic.

SNS then delivers that message to subscribed endpoints or services.

For example:

Suppose an order is successfully placed in our application.

The application publishes an order notification to an SNS topic.

SNS can then send that message to SQS, Lambda, email, or other subscribers.

Example:

```
Application
   ↓
SNS Topic
 ├── SQS
 ├── Lambda
 └── Email
```

Simple ga gurthupettuko:

SNS = One message → Multiple subscribers

Topics

2. What is an SNS Topic?

Answer:

An SNS Topic is a communication channel where publishers send messages.

Subscribers subscribe to the topic and receive messages published to it.

A single topic can have multiple subscribers.

For example:

Suppose we create an OrderNotifications topic for our shopping application.

When an order is placed, the application publishes one message to this topic.

SNS delivers that message to all configured subscribers.

Example:

```
Publisher
   ↓
SNS Topic
   ↓
_____________
↓    ↓    ↓
SQS  Lambda  Email
```

Simple ga gurthupettuko:

Topic = Place where messages are published

---

3. What is Publish-Subscribe in SNS?

Answer:

Publish-subscribe means a publisher sends a message without directly communicating with each subscriber.

The publisher sends the message to an SNS topic.

SNS takes responsibility for delivering the message to the subscribed endpoints.

For example:

Suppose our application needs to notify three different systems about a payment.

The application publishes only one message to SNS.

SNS distributes that message to all required subscribers.

Example:

Publisher
↓
SNS Topic
├— Payment Service
├— SQS
└— Notification Service
Simple ga gurthupettuko:
Pub/Sub = Publisher → Topic → Subscribers

Subscriptions
4. What is an SNS Subscription?
Answer:
A subscription defines an endpoint that receives messages from an SNS topic.
An SNS topic can have multiple subscriptions.
The endpoint can be an AWS service or another supported notification destination.
For example:
Suppose an SNS topic has SQS, Lambda, and email subscriptions.
Whenever a message is published to the topic, SNS attempts to deliver it to those subscribers.
Each subscriber can process the notification according to its purpose.
Example:
SNS Topic
↓
Subscriptions
├— SQS
├— Lambda
└— Email
Simple ga gurthupettuko:
Subscription = Who receives the topic message

---

5. What is the difference between SNS Topic and Subscription?
Answer:
A Topic is the central channel where messages are published.
A Subscription defines where those messages should be delivered.
One topic can have many subscriptions.
For example:
We create one OrderTopic for order notifications.
SQS, Lambda, and email can subscribe to that topic.
The application only needs to publish the message once.
Example:
Topic
↓
├— Subscription → SQS
├— Subscription → Lambda
└— Subscription → Email
Simple ga gurthupettuko:
Topic = Message channel
Subscription = Message receiver

Message Publishing
6. How do you publish a message to an SNS Topic?
Answer:
A publisher sends a message to the SNS topic using the AWS Console, AWS CLI, SDK, or API.
SNS receives the message and identifies the subscriptions associated with the topic.
It then delivers the message to the configured subscribers.
For example:
Suppose our application creates a new order.
It publishes an Order Created message to the SNS topic.
SNS then sends that notification to the subscribed services.
Example:
Application
↓
Publish Message
↓
SNS Topic
↓
Subscribers
Simple ga gurthupettuko:
Publish = Send message to Topic

Access Policy
7. What is an SNS Topic Access Policy?

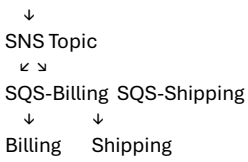
Answer:
An SNS access policy controls who can perform actions on an SNS topic.
It can allow or deny operations such as publishing or subscribing based on configured permissions.
This helps secure the topic from unauthorized access.
For example:
Suppose only our application role should publish messages to an SNS topic.
We can configure permissions so that the required role can publish while unauthorized identities cannot.
This protects the notification system.
Example:
Application Role
↓
SNS Access Policy
↓
SNS Topic
↓
Allow / Deny
Simple ga gurthupettuko:
Access Policy = Who can access SNS Topic

Encryption
8. What is SNS Topic Encryption?
Answer:
SNS supports server-side encryption to protect messages stored within the service.
AWS KMS keys can be used to control encryption for supported SNS topics.
Encryption helps protect sensitive message data.
For example:
Suppose our SNS topic contains sensitive order or customer information.
We can enable server-side encryption using an appropriate KMS key.
SNS then protects the message data using encryption.
Example:
Application
↓
SNS Topic
↓
KMS Encryption
↓
Protected Message
Simple ga gurthupettuko:
SNS Encryption = Protect SNS messages

Delivery Status
9. What is SNS Delivery Status?
Answer:
SNS delivery status information helps us understand whether messages were successfully delivered to supported endpoints.
It is useful for monitoring notification delivery and troubleshooting failures.
The available delivery-status features depend on the endpoint type.
For example:
Suppose SNS sends notifications to a subscriber and some deliveries fail.
We can use the available delivery status information and monitoring data to investigate the problem.
This helps identify delivery-related issues.
Example:
SNS Topic
↓
Message
↓
Subscriber
├— Delivered
└— Failed
Simple ga gurthupettuko:
Delivery Status = Check message delivery

SQS Integration
10. How does SNS work with SQS?
Answer:
SNS can send messages to one or more SQS queues through subscriptions.
This creates a fan-out architecture where one published message can reach multiple queues.
SQS then stores the messages until consumers process them.
For example:
Suppose an order event needs to be processed by billing and shipping separately.
We can subscribe two SQS queues to the same SNS topic.
One order message can then reach both queues.

Example:



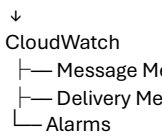
Simple ga gurthupettuko:
SNS = Fan-out
SQS = Store and process messages

Lambda Integration
11. How does SNS work with Lambda?
Answer:
SNS can invoke a Lambda function when a message is published to a topic.
The Lambda function receives the SNS event and performs the required processing.
This is useful for event-driven serverless applications.
For example:
Suppose an image-processing application publishes an event to SNS.
SNS triggers a Lambda function through its subscription.
Lambda can then process the event automatically.
Example:



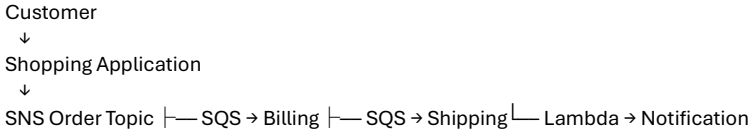
Simple ga gurthupettuko:
SNS + Lambda = Event-driven processing

Monitoring
12. How do you monitor SNS?
Answer:
SNS provides monitoring metrics through Amazon CloudWatch.
We can monitor information such as published messages and delivery-related metrics.
CloudWatch alarms can be used when we need alerts for important SNS activity.
For example:
Suppose our application normally publishes thousands of messages but suddenly message publishing drops.
We can check SNS CloudWatch metrics to identify the change.
This helps us detect application or notification problems.
Example:



Simple ga gurthupettuko:
SNS Monitoring = CloudWatch metrics + alarms

Most Important Interview Scenario
13. Explain a real-time SNS architecture.
Answer:
An application publishes an event to an SNS topic when something important happens.
SNS distributes that event to multiple subscribers such as SQS queues or Lambda functions.
Each subscriber can independently process the same event.
For example:
Suppose a customer places an order in our shopping application.
The application publishes an order event to SNS, which sends it to billing, shipping, and notification services.
Each service processes the same order event independently.
Example:



Simple ga gurthupettuko:
SNS = One event → Multiple services